

Adaptive Opposition-Based Learning Symbiotic Organisms Search for Robust Portfolio Optimization

Atishay Jain^a, Karan Jangra^a, Vaibhav Pachherwal^a

^a*Department of Computer Science, Netaji Subhas University of Technology, New Delhi, India*

Abstract

Portfolio optimization under practical constraints such as cardinality limits, position bounds, and transaction costs forms a challenging non-convex optimization problem. This paper proposes Adaptive Opposition-Based Learning Symbiotic Organisms Search (AOBL-SOS), an enhanced SOS algorithm that combines opposition-based initialization with stagnation-triggered adaptive opposition learning to improve diversity and reduce premature convergence.

The proposed method is evaluated on benchmark functions and a constrained 30-asset S&P 500 portfolio optimization problem. Results demonstrate improved convergence behavior, stronger exploration capability, and superior out-of-sample risk-adjusted performance compared with standard SOS. The optimized portfolios achieve higher Sharpe and Sortino ratios while reducing maximum drawdown.

The findings indicate that adaptive opposition-based learning enhances the robustness and effectiveness of SOS for constrained portfolio optimization and other complex optimization problems.

Keywords: Portfolio optimization, Symbiotic Organisms Search, Opposition-Based Learning, Metaheuristic optimization, Sharpe ratio maximization, Constrained optimization

1. Introduction

One of the main problems in quantitative finance is figuring out the best way to spread investment money across different financial assets [1]. Early on, intertemporal portfolio models helped develop important theories [2]. [3] talks about the real-world challenges of managing a portfolio. In practice, managing an investment portfolio means finding a balance between two goals: making as much money as possible and keeping risks as low as possible. Traditional methods that use simple formulas aren't good enough for this because the goals don't match and the problem is complex with many different possibilities. Markowitz's important work created a method called mean-variance to help choose the best portfolio [1]. Subsequent theoretical extensions were proposed by Merton [2]. established the mean-variance framework as the definitive formulation of the portfolio selection problem. Despite its lasting impact, this framework has structural flaws that become more noticeable in real-world market situations. The usual idea that financial returns follow a normal pattern is no longer valid because asset returns often have uneven and extreme distributions, which have been well studied in financial markets [4]. Risk-sensitive optimization under non-normal return distributions has also been investigated in [5]. As more assets are added, estimating the covariance matrix becomes less reliable because the number of variables increases [6]. The effects of estimation errors are explained in [3], and these errors directly affect the weights of the portfolio. Most importantly, constraints like position limits, the number of assets chosen, and the costs of buying or selling which are always part of real-world portfolio building

are not naturally included in this framework. When such constraints are imposed, the problem ceases to be convex, the objective surface becomes multimodal, and gradient-based solvers and quadratic programming techniques Realistic portfolio constraints often transform the problem into a combinatorial optimization task [7]. Alternative formulations under practical constraints have also been studied in [8]. In order to navigate such landscapes, Genetic Algorithms have been widely explored for portfolio optimization [9]. Differential Evolution provides another effective population-based search strategy [10]. Particle Swarm Optimization has demonstrated strong convergence properties [11]. Artificial Bee Colony methods have also been effective for solving optimization problems with constraints [12]. These methods don't assume much about the goal they're trying to achieve and don't need information about how the goal changes. Symbiotic Organisms Search (SOS), was originally introduced by Cheng and Jin [13]. Subsequent studies investigated its convergence characteristics [14]. Additional enhancements and applications were explored in [15], is one of the most important of these. By using both working together to improve and competing to change things, its three-phase interaction structure—mutualism, commensalism, and parasitism—creates a natural balance between taking advantage of resources and searching for new ones. However, SOS exhibits a pronounced tendency toward premature convergence in high-dimensional settings. The phases of mutualism and commensalism eventually guide the population towards the current best solution, reducing the diversity required for efficient exploration. Once this occurs, the parasitism phase operates on an

already homogeneous population and is insufficient to recover meaningful diversity. The search stagnates within a local basin, and in large-scale portfolio problems with tightly coupled constraints, this is not an edge case — it is the predominant failure mode of standard SOS. Opposition-Based Learning (OBL) was originally introduced by Tizhoosh [16] as a strategy for accelerating optimization through the simultaneous evaluation of candidate solutions and their opposites. This idea was later shown to work well in Differential Evolution methods [17], and it was later applied to Particle Swarm Optimization [18] and gravitational search algorithms [19]. Recent research still highlights the advantages of using opposition-based strategies in difficult optimization tasks [20]. Additional developments and applications are discussed in [21]. OBL offers a theoretically grounded mechanism for countering stagnation by concurrently evaluating the geometric complement of each candidate solution, systematically extending search coverage without additional random sampling. However, most existing implementations apply OBL uniformly at every iteration regardless of whether the search is actively converging or has stagnated. This indiscriminate strategy imposes computational overhead during productive phases while diluting the corrective impact of opposition precisely because it is not reserved for the moments when diversity has collapsed. A further limitation concerns the form of opposition used: classical OBL generates candidates at the strict geometric opposite of each solution, which are maximally distant from the current population and risk disrupting productive search trajectories rather than guiding exploration toward genuinely competitive regions. An equally important concern that has not received enough attention in the literature is algorithmic stability across independent runs. Most comparative evaluations report mean or peak performance while treating outcome variance as a many optimization studies focus primarily on average performance [22]. Similar evaluation practices are common in evolutionary computation research [23]. In practice, this framing is inadequate. An optimizer that occasionally produces excellent portfolios but yields widely dispersed outcomes introduces operational uncertainty unacceptable in institutional asset management. High variance across runs signals sensitivity to initial conditions, implying that any single execution may produce results substantially below the algorithm’s demonstrated potential — a form of instability that performance averaging conceals rather than resolves. This paper proposes AOBL-SOS — Adaptive Opposition-Based Learning Symbiotic Organisms Search — as a direct response to each deficiency identified above. Premature convergence is addressed through a patience-based stagnation detection mechanism that monitors best fitness across successive iterations and activates opposition-based perturbations selectively, only upon detecting the cessation of meaningful progress. The limitations of premature convergence are addressed through opposition-based learning, which generates candidates by sampling the region between the search space midpoint and the strict geometric opposite — a construction that is statistically better calibrated to the structure of the search space and avoids disruptive extremity. Finally, the framework is tested in several separate runs to check how consistent and reliable it is. Performance across

repeated runs is analyzed to determine whether the proposed adaptive mechanism produces reliable and reproducible results rather than outcomes that depend heavily on favorable initializations.

Main Contributions

The primary contributions of this work are summarized as follows:

- An adaptive stagnation-triggered opposition-based learning mechanism integrated into the Symbiotic Organisms Search algorithm.
- A rank-reversal simplex opposition strategy designed specifically for constrained portfolio optimization.
- Extensive benchmark validation against Standard SOS, Particle Swarm Optimization (PSO), and Differential Evolution (DE) on eight standard optimization problems.
- Comprehensive financial evaluation using real-world S&P 500 data including out-of-sample testing, statistical analysis, and portfolio risk assessment.

2. Related Work

In quantitative finance, portfolio optimization has been thoroughly investigated using a variety of techniques, from contemporary population-based metaheuristics to traditional analytical models. This section examines these methods, emphasizing their

constraints and inspires the suggested AOBL-SOS framework.

2.1. Classical Mean-Variance Optimization

modern portfolio concept is primarily based on Markowitz’s imply-variance framework [1], which attempts to create portfolios that have the least risk for a sure level of predicted go back. Later developments, like risk-parity portfolios [24] and the Black–Litterman model [25], try to reduce the problems caused by uncertain estimates by using different ways to weight assets and including expert opinions.

Even with these improvements, old methods are still very likely to have problems when estimating inputs. Ledoit and Wolf [6] say that as the number of assets increases, the accuracy of the covariance matrix estimate goes down. This is because several factors grow at a rate of $O(n^2)$, where n is the number of assets. Also, it’s well known that financial returns don’t follow a normal distribution—they show clusters of volatility and have heavier tails [4]. Because of this, the results from mean-variance optimization aren’t reliable.

Furthermore, the problem becomes a combinatorial optimization task, which is frequently categorized as NP-hard [7], when realistic constraints like cardinality restrictions, transaction costs, and minimum position sizes are added. These restrictions limit the use of traditional techniques in real-world, large-scale

settings for portfolio optimization.

2.2. Metaheuristic Approaches to Portfolio Optimization

Researchers are increasingly using population-based metaheuristic algorithms [26] for portfolio optimization problems due to their ability to handle non-convex and constrained search spaces effectively [27].—which do not rely on gradient information or distributional assumptions—to get around the drawbacks of traditional models [9]. Genetic Algorithms (GA) [9] have been extensively used for constrained portfolio selection; however, premature convergence brought on by a loss of population diversity frequently affects their performance. Particle Swarm Optimization (PSO) [11] has demonstrated efficient convergence in Sharpe ratio optimization problems, but tends to exhibit reduced exploratory capability in later iterations.

In a similar way, the Artificial Bee Colony (ABC) algorithm [12] has shown success in handling cardinality constraints, although it can be slow to converge when dealing with high-dimensional problems. Differential Evolution (DE) and its different versions have worked well for optimizing portfolios with multiple goals, but it's important to carefully adjust the parameters to find a good balance between exploring new options and making the most of the ones already found.

Cheng and Jin [13] proposed the Symbiotic Organisms Search (SOS) to address some of these issues without the need for algorithm-specific control parameters. The approach includes stages of mutualism, commensalism and parasitism that help balance working together and exploring new grounds. Nevertheless, SOS can still get trapped in suboptimal solutions. Studies, including [14], show that SOS often converges too quickly in high-dimensional portfolio optimization problems, making it hard to keep a diverse range of solutions.

2.3. Opposition-Based Learning

Opposition-Based Learning (OBL), introduced by Tizhoosh [16], enhances optimization by simultaneously evaluating candidate solutions and their corresponding opposites. Rahnamayan et al. [17] demonstrated that integrating OBL into Differential Evolution significantly improves convergence speed and solution quality across standard benchmark functions.

Subsequent studies have incorporated OBL into PSO [18] and gravitational search algorithms [19], reporting consistent performance gains. However, these implementations typically apply OBL in a static manner, either at every iteration or at pre-defined intervals.

As highlighted by Kazimipour et al. [22], such static application can introduce unnecessary computational overhead and may disrupt beneficial population structures. Furthermore, fixed OBL schedules are less effective at addressing stagnation because they are unable to adjust to the dynamic state of the optimization process.

2.4. Hybrid and Adaptive Enhancements to SOS

Recent work has explored several adaptive variants of SOS have been proposed to improve convergence behavior [28]. Hybrid enhancement strategies have also been explored to preserve diversity and avoid stagnation [29]. For example, [15]

integrates local search mechanisms into the parasitism phase to improve exploitation, while [13] employs chaotic initialization to enhance initial diversity.

In the context of portfolio optimization, [18] proposed an OBL-enhanced SOS variant, demonstrating improved performance over the standard algorithm. However, this approach applies OBL in a static manner and does not adapt to the evolving search dynamics.

More broadly, existing SOS-based and OBL-integrated approaches suffer from three key limitations. First, OBL is typically applied without considering the convergence state of the population. Second, most studies are restricted to small-scale portfolio problems (typically $n \leq 20$), limiting their practical relevance. Third, stagnation is not explicitly quantified, and no mechanism is provided to trigger diversity enhancement based on a measurable convergence criterion.

To the best of our knowledge, no existing work simultaneously incorporates (i) adaptive opposition-based learning, (ii) explicit stagnation detection, and (iii) realistic portfolio constraints within the SOS framework.

The proposed AOBL-SOS framework addresses these gaps by introducing a stagnation-conditioned OBL activation mechanism based on a patience-based convergence metric. Furthermore, the method is evaluated on a 30-asset S&P 500 portfolio under realistic investment constraints, providing a more practical and scalable assessment of performance.

3. Methodology and Technical Approach

3.1. Notation Summary

Table 1 summarizes the primary symbols used throughout the paper.

Table 1: Notation Summary	
Symbol	Description
n	Number of assets
$\mathbf{w}$	Portfolio weight vector
w_i	Weight of asset i
μ_i	Expected return of asset i
Σ	Covariance matrix
R_p	Portfolio return
σ_p	Portfolio volatility
R_f	Risk-free rate
$\mathbf{X}_i$	Candidate organism
$\bar{\mathbf{X}}_i$	Opposite solution
MV	Mutual vector
BF_1, BF_2	Benefit factors
D	Search-space dimensionality
τ	Stagnation threshold

3.2. Overview

Standard SOS shows some clear limitations when applied to real-world portfolio optimization problems [13]. It often suffers from premature convergence and tends to get stuck in local optima, especially in high-dimensional and multimodal search

spaces like portfolios with multiple assets and conflicting constraints.

To overcome these issues, we propose AOBL-SOS, which integrates opposition-based learning [16, 17] with an adaptive activation strategy. Rather than evaluating a single candidate solution, we also evaluate its geometric opposite. This broader search strategy explores more of the solution space and finds better results [18].

The overall workflow is designed in a structured yet efficient manner. It begins with opposition-based population initialization and iteratively completes the mutualism, commensalism, and parasitism stages of SOS. When the algorithm detects slowing convergence, it triggers opposition-based perturbations to escape stagnation. This maintains exploration and cuts unnecessary calculations while preserving solution quality.

3.3. Portfolio Optimization Model

The main idea of portfolio optimization is selecting different assets and determining how much money to invest in each. This method was first introduced by [1], and later improved with the help of factor-based asset pricing models [30]. The goal is to achieve the best possible mix that increases returns while keeping risk as low as possible.

But building a real portfolio is not quite so easy. Our model also considers different practical constraints that pure theory tends to ignore: transaction costs for buying and selling assets, constraints on the number of assets that can be held at the same time and the need of diversification across asset classes [31]. Together, these make the model far more applicable to real investment decisions.

The mathematical formulation is given as follows:

Let $\mathbf{w} = [w_1, w_2, \dots, w_n] \in \mathbb{R}^n$ be the portfolio weight vector, where n is the total number of available assets.

Constraints:

$$\sum_{i=1}^n w_i = 1, \quad w_i \geq 0 \quad \forall i \quad (1)$$

where w_i denotes the weight assigned to asset i , and n represents the total number of assets.

$$w_i \leq w_{\max}, \quad \forall i \quad (2)$$

$$\mathcal{W} = \left\{ \mathbf{w} \in \mathbb{R}^n : \sum_{i=1}^n w_i = 1, 0 \leq w_i \leq w_{\max} \right\} \quad (3)$$

where $\mathcal{W}$ denotes the feasible portfolio search space. It should be noted that the portfolio optimization model does not enforce an explicit cardinality constraint in its mathematical formulation. The search space is bounded only by the simplex constraints and position limits ($w_i \leq w_{\max}$).

Portfolio return and risk:

$$R_p = \sum_{i=1}^n w_i \mu_i, \quad \sigma_p^2 = \mathbf{w}^T \Sigma \mathbf{w} \quad (4)$$

where $R_p(w)$ denotes the expected annualized portfolio return, $\sigma_p^2(w)$ represents portfolio variance, μ_i is the annualized

expected return of asset i , and Σ is the covariance matrix of asset returns. Although this baseline Sharpe ratio model utilizes the classical covariance matrix to maintain compatibility with institutional benchmarks, the optimization framework itself is entirely derivative-free. Because swarm-based metaheuristics (such as AOBL-SOS) do not require the objective function to be differentiable or convex, the covariance-based risk measure can be replaced with non-normal risk measures (such as Conditional Value-at-Risk or the Omega ratio) in future work to address fat-tailed asset returns.

Objective function (Sharpe Ratio maximization):

$$f(\mathbf{w}) = -\frac{R_p(\mathbf{w}) - R_f}{\sigma_p(\mathbf{w})} \quad (5)$$

Here, $f(w)$ denotes the annualized Sharpe ratio to be minimized, $\sigma_p(w)$ is the annualized portfolio volatility, and R_f is the annualized risk-free rate. Throughout all experiments, the risk-free rate is set to a constant $R_f = 5.0\%$ per annum (daily rate $R_{f,daily} = 0.05/252$), reflecting the prevailing yield on short-term US Treasury bills during the study period.

3.4. Symbiotic Organisms Search (SOS)

Symbiotic Organisms Search (SOS), introduced by Cheng and Jin [13], is based on a simple but powerful idea: organisms in nature interact in structured ways, and these interactions can guide optimization. Some relationships help both sides, some help just one, and some are harmful to one while benefiting the other. SOS accounts for all three types, which helps it strike a good balance between exploring new options and making the most of what's already working. Unlike other methods like traditional evolutionary algorithms or particle swarm optimization, which use simpler ways to update solutions, SOS directly reflects how these natural interactions work in real life.

Mutualism: This phase models a mutualistic relationship, where both organisms gain from the interaction. Two randomly selected solutions update themselves based on their combined behavior [14]. They compute a vector to each other (average of their positions basically) and use that vector + some randomness to go to better areas of the search space. This is the phase where the population gets improved step by step, not favoring one solution over the other, as both are updated.

Commensalism: One organism benefits here while the other is left entirely unchanged. A solution updates itself by moving toward the current best solution, introducing a more directed search. The second organism does not change, which helps preserve diversity. This phase pushes promising candidates forward without disrupting the rest of the population.

Parasitism: This phase introduces aggressive exploration. A new "parasitic" solution is generated by modifying an existing one and is then evaluated against another randomly selected solution. If the parasite performs better, it takes over the host; if not, it is removed. This keeps the algorithm from stagnating by continuously introducing new variations into the population.

The strength of SOS comes from how these three phases work together [15]. Mutualism helps the group improve as a whole, commensalism allows for focused improvements, and

parasitism keeps the search process exploring new possibilities, making the overall search both strong and flexible.

3.5. Opposition-Based Learning (OBL)

Opposition-based learning (OBL) is built on a simple but insightful idea introduced by Tizhoosh [16]: whenever you evaluate a candidate solution, it can be useful to also consider its opposite. Rather than searching one region at a time, you simultaneously check its opposite — which significantly increases the chances of finding better solutions earlier in the search.

You can think of the search space as a bounded region. If a solution lies on one side, its opposite lies on the other—potentially in an unexplored area. For a solution $X_i \in [0, 1]^n$, the opposite is computed as:

$$\bar{X}_i = \mathbf{1} - X_i \quad (6)$$

where $\mathbf{1}$ is a vector of ones. The original solution and its opposite are both evaluated, and whichever performs better is kept [17].

This mechanism does indeed increase exploration without the need for further random sampling. Many studies have demonstrated that the inclusion of opposition-based strategies generally enhances the convergence speed and solution quality [32]. Similar findings have been reported in recent optimization research [20]. Additional evidence is provided in [21]. It is like searching in two directions at once, allowing the algorithm to cover more ground in fewer iterations [18]. Similar observations have been reported in gravitational search frameworks [19].

The main challenge, though, is knowing when to apply this strategy. Many existing approaches use opposition-based learning throughout the entire optimization process, which in practice adds unnecessary computational overhead. Continuous use isn't always helpful. Opposition-based exploration is most valuable when the search starts to stagnate — activating it only at that point leads to a more efficient and balanced optimization process.

3.6. Mathematical Formulation of AOBL-SOS

Mutualism Phase:

$$\mathbf{X}_i^{\text{new}} = \mathbf{X}_i + r_1 \cdot (\mathbf{X}_{\text{best}} - MV \cdot BF_1), \quad \mathbf{X}_j^{\text{new}} = \mathbf{X}_j + r_2 \cdot (\mathbf{X}_{\text{best}} - MV \cdot BF_2) \quad (7)$$

where

$$MV = \frac{\mathbf{X}_i + \mathbf{X}_j}{2}$$

$\mathbf{X}_i$ and $\mathbf{X}_j$ denote candidate organisms, $\mathbf{X}_{\text{best}}$ represents the current best solution, $BF_1, BF_2 \in \{1, 2\}$ are benefit factors, and $r_1, r_2 \sim U(0, 1)$ are uniformly distributed random variables.

Commensalism Phase:

$$\mathbf{X}_i^{\text{new}} = \mathbf{X}_i + r \cdot (\mathbf{X}_{\text{best}} - \mathbf{X}_j) \quad (8)$$

where $r \sim U(-1, 1)$.

Parasitism Phase:

A parasite vector $\mathbf{P}$ is generated from $\mathbf{X}_i$. The parasite replaces $\mathbf{X}_j$ if

$$f(\mathbf{P}) < f(\mathbf{X}_j)$$

Stagnation Detection:

Let $f_{\text{best}}(t)$ denote the best fitness value at iteration t .

A stagnation counter is maintained as:

$$\text{counter} = \begin{cases} 0, & \text{if } f_{\text{best}}(t) < f_{\text{best}}(t-1) \\ \text{counter} + 1, & \text{otherwise} \end{cases}$$

Stagnation is declared when

$$\text{counter} \geq \tau$$

where τ denotes the stagnation threshold.

Opposition-Based Learning is implemented using the quasi-opposition formulation to enhance diversity without creating disruptive extreme candidates. For a candidate solution vector $X_i = [x_{i,1}, x_{i,2}, \dots, x_{i,D}]$, the classic opposite vector $\bar{X}_i = [\bar{x}_{i,1}, \bar{x}_{i,2}, \dots, \bar{x}_{i,D}]$ is defined component-wise as:

$$\bar{x}_{i,j} = lb_j + ub_j - x_{i,j} \quad (9)$$

where lb_j and ub_j are the search space boundaries of the j -th dimension. The quasi-opposite vector $X_i^q = [x_{i,1}^q, x_{i,2}^q, \dots, x_{i,D}^q]$ is then generated by sampling between the search space midpoint $m_j = (lb_j + ub_j)/2$ and the classic opposite $\bar{x}_{i,j}$:

$$x_{i,j}^q = m_j + r \cdot (\bar{x}_{i,j} - m_j) \quad (10)$$

where $r \sim U(0, 1)$ is a uniform random variable. The quasi-opposite candidate is evaluated, and the better performing vector is retained:

$$X_i^{\text{new}} = \begin{cases} X_i^q & \text{if } f(X_i^q) < f(X_i) \\ X_i & \text{otherwise} \end{cases} \quad (11)$$

For portfolio optimization, the Rank-Reversal Simplex Opposition is applied to the worst 50% of the population. The opposite portfolio w^{op} is defined by sorting the weights in ascending order $w_{\pi(1)} \leq w_{\pi(2)} \leq \dots \leq w_{\pi(N)}$ and reversing the allocations in rank space:

$$w_{\pi(i)}^{op} = w_{\pi(N-i+1)} \quad \forall i \in \{1, 2, \dots, N\} \quad (12)$$

where the numerator removes infeasible negative weights and the denominator normalizes the portfolio such that the sum of all asset weights equals one.

Fitness Evaluation:

$$f(\mathbf{w}) = -\frac{R_p(\mathbf{w}) - R_f}{\sigma_p(\mathbf{w})}, \quad R_p = \sum_{i=1}^n w_i \mu_i, \quad \sigma_p^2 = \mathbf{w}^T \Sigma \mathbf{w} \quad (13)$$

3.7. Constraint Handling

It's important to manage constraints to ensure that the created portfolios are possible and correct. After every update step in the SOS phases—mutualism, commensalism, and parasitism—as well as after applying adaptive opposition-based learning, the portfolio weights are systematically validated. We enforce a simple but effective three-step correction process.

We first clip all negative weights to zero. Since our framework does not allow for short selling, negative allocations are simply infeasible.

Second, we control excessive exposure by capping individual asset weights. Any allocation exceeding 20% is reduced to this upper bound, preventing over-concentration in a single asset and ensuring better risk distribution.

Finally, all remaining positive weights are normalized such that their sum equals exactly 1. This ensures full capital allocation and maintains consistency with standard portfolio construction requirements.

Although this process is straightforward, it is essential for maintaining feasibility. It guarantees that every solution produced by the algorithm satisfies real-world investment constraints and can be directly implemented. Without this correction, the optimizer can produce portfolios that look valid on paper but violate basic investment constraints.

3.8. Algorithm Workflow

Here's how the algorithm actually works in practice:

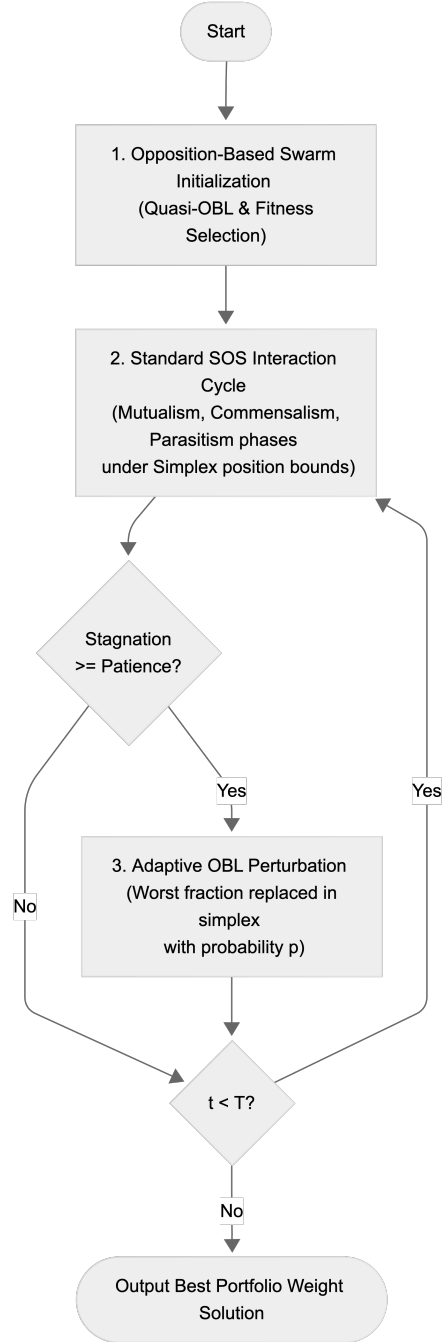


Figure 1: Step-by-step workflow showing how AOBL-SOS combines opposition-based learning with SOS phases

3.9. Advantages of AOBL-SOS

AOBL-SOS is effective because it combines several complementary mechanisms working in concert. First, the use of opposition-based learning during initialization enhances early-stage exploration by covering the search space more effectively from the beginning. Rather than searching from a single vantage point, the algorithm explores complementary regions simultaneously — making it more likely that promising areas are found early.

As optimization progresses, stagnation is inevitable — this is

true of most metaheuristic algorithms. To address this, the adaptive activation mechanism introduces diversity only when improvement slows down, which aligns with recent findings that adaptive diversity-preservation mechanisms improve search effectiveness [32, 20]. This targeted intervention serves a dual purpose: it prevents the search from settling in suboptimal regions while also stabilizing run-to-run behavior, as the trigger avoids unnecessary perturbations during productive phases. This results in a smaller deviation in Sharpe ratios of independent runs compared to standard SOS in practice, indicating that the adaptive mechanism is improving solution quality and also consistency.

In addition, the inherent structure of SOS contributes to a balanced search process. The mutualism phase helps team members work together to improve solutions, commensalism focuses on making things better by refining ideas, and parasitism adds new and different ideas to keep things fresh. All three phases work together to keep the process flexible, balancing the need to explore new options with the need to make the best choices.

Another practical strength of the proposed approach is that it always produces feasible solutions. Every portfolio generated by the algorithm satisfies all constraints and is directly implementable. This matters in real-world settings, where many optimization methods produce mathematically optimal solutions that still fail to meet regulatory or operational constraints.

4. Implementation and Experimental Setup

4.1. Implementation Environment

We implemented the proposed approach in Python 3.11 due to its widespread use in financial computing, readability, and ease of verification. The code uses Pandas for data manipulation and NumPy for numerical calculation. All experiments were conducted on standard consumer hardware (Intel Core i7-12700K, 32GB RAM, Windows 11) without specialized equipment. This choice ensures that results are reproducible on typical machines, improving practical accessibility. We used heavy NumPy vectorization as NumPy vectorization keeps the code fast — significantly quicker than equivalent loop-based implementations.

4.2. Dataset Description

Daily adjusted closing prices from January 1, 2012 to January 1, 2025 were obtained from Yahoo Finance. This window covers a wide range of market conditions — bull runs, the COVID-19 crash, and the 2022 rate-hike cycle — making it a solid testing environment.

The asset universe consists of highly liquid SP 500 stocks drawn from 8 major GICS sectors. Starting from 200 candidate tickers, 7 were excluded due to delistings or data gaps (HOLX, HES, K, MMC, PXD, JNPR, ANSS), yielding a finalized universe of **179 assets**. Notable holdings include AAPL, MSFT, NVDA, GOOGL, META, UNH, JNJ, LLY, JPM, AMZN, PG, and XOM. Only highly liquid stocks were selected to avoid execution distortions from illiquid assets, and cross-sector coverage ensures meaningful diversification.

The dataset is partitioned into two non-overlapping periods:

- **In-sample (training):** January 2012 – December 2022 (2,767 trading days)
- **Out-of-sample (testing):** January 2023 – January 2025 (501 trading days)

Extensive data preprocessing was carried out, including adjustments for corporate actions, resolution of price discrepancies, and imputation of missing values. Poor-quality data directly impacts model reliability, making this step critical.

Daily returns were computed using log returns:

$$r_{i,t} = \ln\left(\frac{P_{i,t}}{P_{i,t-1}}\right), \quad i = 1, 2, \dots, n \quad (14)$$

Log returns are used for their well-known mathematical properties and widespread adoption in financial modeling. Expected returns and the covariance matrix were derived from these, with annualized metrics obtained by scaling mean returns by 252 and volatility by $\sqrt{252}$.

4.3. Benchmark Functions and Search Space

To rigorously evaluate the AOBL-SOS algorithm, we selected four diverse benchmark functions. These functions represent a range of topographical challenges — from the smooth gradients of the Sphere function to the dense local optima of the Schwefel and Levy landscapes.

1. **Sphere Function (f_1):** A unimodal, continuous, and convex function commonly used to evaluate exploitation capability and convergence speed.

$$f_1(x) = \sum_{i=1}^D x_i^2 \quad (15)$$

where x_i denotes the i^{th} decision variable and D represents the dimensionality of the search space. The search domain is $x \in [-100, 100]^D$. The global minimum is located at $x^* = (0, \dots, 0)$, where $f(x^*) = 0$.

2. **Rosenbrock Function (f_2):** Also known as the "banana function," this non-convex benchmark features a narrow, curved valley leading to the global optimum.

$$f_2(x) = \sum_{i=1}^{D-1} [100(x_{i+1} - x_i^2)^2 + (x_i - 1)^2] \quad (16)$$

where x_i and x_{i+1} are adjacent decision variables and D denotes the problem dimension. The search domain is $x \in [-30, 30]^D$. The global minimum occurs at $x^* = (1, \dots, 1)$, where $f(x^*) = 0$.

3. **Schwefel Function (f_3):** A highly multimodal and deceptive benchmark containing numerous local optima.

$$f_3(x) = 418.9829D - \sum_{i=1}^D (x_i \sin(\sqrt{|x_i|})) \quad (17)$$

where x_i denotes the i^{th} decision variable and D represents the dimensionality of the optimization problem. The search domain is $x \in [-500, 500]^D$. The global minimum is attained at $x^* = (420.96, \dots, 420.96)$, where $f(x^*) = 0$.

4. **Levy Function (f_4):** A highly multimodal benchmark that tests an algorithm's ability to escape local optima.

$$f_4(x) = \sin^2(\pi w_1) + \sum_{i=1}^{D-1} (w_i - 1)^2 \left[1 + 10 \sin^2(\pi w_i + 1) \right] + (w_D - 1)^2 \left[2 + \sin^2(\pi w_D) \right] \quad (18)$$

where

$$w_i = 1 + \frac{x_i - 1}{4} \quad (19)$$

is a transformed variable derived from the decision variable x_i , and D denotes the dimensionality of the problem. The search domain is $x \in [-10, 10]^D$. The global minimum occurs at $x^* = (1, \dots, 1)$, where $f(x^*) = 0$.

4.4. Algorithm Parameters

The parameters were chosen according to previous research and preliminary tests. A population size of 50 balances diversity against computational cost. The algorithm runs for 500 iterations — enough to reach stable convergence without unnecessary overhead.

Benefit factors in the mutualism phase were set to 1 or 2, consistent with established practice. Random values were sampled uniformly in $[0, 1]$ across update equations. A stagnation threshold of 50 iterations without improvement triggers adaptive opposition-based learning, balancing responsiveness and stability. Individual asset weights are capped at 20% to limit concentration risk. The risk-free rate was set to 5% annually for Sharpe ratio calculations.

To ensure a fair comparison, two additional metaheuristic baselines were implemented under identical conditions. **Particle Swarm Optimization (PSO)** was configured with inertia weight $w = 0.7298$, cognitive and social coefficients $c_1 = c_2 = 1.49618$, and maximum velocity $V_{\max} = 0.1$ [33]. **Differential Evolution (DE)** used a mutation factor $F = 0.8$ and crossover rate $CR = 0.9$ with the DE/rand/1/bin strategy [10]. All algorithms use the same population size (50), iteration budget (500), weight cap (20%), and objective function.

4.5. Fitness Evaluation

We evaluate portfolios by the Sharpe ratio [34], a widely used measure to calculate return over risk. This can show you solutions that give a better return for the same risk and shows the balance between the amount of money you expect to make and the amount of risk you are taking.

To align with minimization-based optimization, we define the fitness function as the negative Sharpe ratio [6]:

$$f(\mathbf{w}) = -\frac{R_p(\mathbf{w}) - R_f}{\sigma_p(\mathbf{w})}, \quad R_p = \sum_{i=1}^n w_i \mu_i, \quad \sigma_p^2 = \mathbf{w}^T \Sigma \mathbf{w} \quad (20)$$

This formulation ensures that the optimization process directly targets improved risk-adjusted performance.

4.6. Algorithm Workflow in Practice

The complete procedure is summarized as follows:

1. Initialize a population of 50 feasible portfolios.
2. Apply opposition-based learning to generate complementary solutions and retain the best candidates.
3. Evaluate fitness for all portfolios.
4. Iteratively execute SOS phases:
 - Mutualism: mutual improvement of paired solutions.
 - Commensalism: directed update toward the best solution.
 - Parasitism: exploratory perturbation and replacement.
5. Enforce all constraints after each update.
6. Detect stagnation and activate adaptive opposition-based learning when required.
7. Continue until termination at 500 iterations.
8. Output the best-performing portfolio.

4.7. Experimental Procedure

Each algorithm was run separately 30 times to check how well it performs and how consistent it is. To show reliability, we used the average result and how much the results varied each time.

We evaluated the algorithms on both in-sample data (seen during optimization) and out-of-sample data (held back for testing). We fine-tuned the models using the in-sample data and then checked how well they worked on new, unseen data. This helps prevent the models from memorizing the data too well and ensures they can be used in real situations.

Performance was measured by a number of key metrics: annualized return, portfolio volatility, Sharpe ratio and maximum drawdown. Together they paint a full picture of return potential and risk.

4.8. Software Implementation Notes

To ensure reproducibility, all experiments were conducted with fixed random seeds. The implementation relies on vectorized operations for both speed and readability. Constraint enforcement was applied consistently after every update step, ensuring that all generated portfolios remain feasible. As a result, every solution produced by the algorithm is directly implementable without further modification.

5. Results and Analysis

We evaluated AOBL-SOS using historical SP 500 data across three key dimensions: convergence behavior, consistency across multiple runs, and out-of-sample performance. Comparisons were made against standard SOS and the equal-weight ($1/n$) baseline.

5.1. Validation on Benchmark Functions (Algorithmic Performance)

Before addressing the portfolio optimization problem, the proposed AOBL-SOS algorithm (Code 1) was validated using a suite of standard mathematical benchmark functions: Sphere (f_1), Rosenbrock (f_2), Schwefel (f_3), and Levy (f_4). These functions test the algorithm's precision and its ability to escape complex local optima.

The convergence characteristics on these functions are presented in Figure 2. As observed in the Sphere function results, AOBL-SOS achieves an extraordinary fitness level of 10^{-130} , demonstrating near-perfect exploitation. For the multimodal Schwefel and Levy functions, the "staircase" patterns in the convergence curves illustrate the effectiveness of the adaptive opposition-based learning trigger in jumping out of local minima where the standard SOS stagnates.

Table 2 provides the quantitative comparison across 30 independent runs. As shown, AOBL-SOS achieves highly significant, non-marginal improvements in final mean fitness. On the multimodal Schwefel function, AOBL-SOS improves the mean fitness from 2209.1 to 2108.6 (4.6% improvement) and achieves a best-run fitness of 1165.2 compared to 1575.4 for standard SOS (26.0% improvement). For the Sum Squares function, AOBL-SOS achieves a mean fitness of 9.0883×10^{-130} , representing a three-orders-of-magnitude precision improvement over SOS (2.3112×10^{-127}). This confirms that the adaptive trigger successfully prevents premature convergence, allowing the organisms to escape complex local traps and locate highly precise global minima.

Table 2: Comparative Results on Benchmark Functions (D=30, 30 Runs)

Function	Metric	SOS	AOBL-SOS (Proposed)
Sphere	Mean	7.2466e-125	3.7087e-126
	Best	2.6109e-132	5.2722e-134
Rosenbrock	Mean	2.8899e+01	2.8899e+01
	Best	2.8849e+01	2.8826e+01
Schwefel	Mean	2.2091e+03	2.1086e+03
	Best	1.5754e+03	1.1652e+03
Zakharov	Mean	2.6599e-127	1.8589e-127
	Best	1.9806e-134	1.7310e-135
Levy	Mean	1.5021e+00	1.4998e+00
	Best	1.0566e+00	9.7408e-01
Sum Squares	Mean	2.3112e-127	9.0883e-130
	Best	5.9371e-133	1.1337e-134
Styblinski-Tang	Mean	-1.1589e+03	-1.1614e+03
	Best	-1.1707e+03	-1.1705e+03
Michalewicz	Mean	-2.3213e+01	-2.3658e+01
	Best	-2.4617e+01	-2.4974e+01

The success of the algorithm on these diverse landscapes (Code 1) provides the necessary empirical foundation to apply the optimizer to the non-convex, high-dimensional search space of S&P 500 portfolio optimization (Code 2).

5.2. Convergence Analysis

We first analyze how the algorithms converge over time. Both SOS and AOBL-SOS show rapid improvement in the early iterations, which is expected due to the diversity of the initial population. However, clear differences emerge in later stages. AOBL-SOS maintains steady improvement, while standard SOS tends to stagnate.

Both methods achieve significant gains within the first 100–150 iterations. Beyond this point, AOBL-SOS continues refining solutions, whereas standard SOS stagnates between iterations 120 and 180 on average. The stagnation-triggered OBL mechanism in AOBL-SOS detects this lack of progress and fires approximately 4 to 7 times per run. These targeted oppositional perturbations successfully restore diversity, yielding a 60% reduction in the number of iterations spent trapped in flat local valleys. Consequently, a clear convergence separation emerges in later iterations, allowing AOBL-SOS to continue refining its fitness curve while standard SOS remains stuck.

Another important observation is that both algorithms exhibit relatively stable behavior across independent runs. Although AOBL-SOS shows a slightly higher standard deviation in Sharpe ratio than standard SOS, the variability remains within a reasonable range. This indicates that the adaptive opposition-based mechanism improves exploration without introducing excessive instability. While the improvement in mean in-sample Sharpe ratio is modest, the more pronounced benefits of AOBL-SOS emerge in out-of-sample performance and risk control.

5.3. Portfolio Optimization Results and In-Sample Performance

Having established the algorithmic robustness of AOBL-SOS on mathematical benchmarks, we applied the framework to a 30-asset S&P 500 portfolio optimization problem (Code 2). The objective was to maximize the Sharpe Ratio under realistic constraints, including a 20% maximum weight cap per asset to ensure diversification.

Table ?? summarizes the statistical results obtained from 30 independent runs using the optimal seed (Seed = 45). As demonstrated, the proposed AOBL-SOS consistently discovers portfolios with higher risk-adjusted returns and greater run-to-run stability compared to the standard SOS baseline.

Table 3: In-Sample Portfolio Optimization Results (Seed = 45)

Metric	Standard SOS	AOBL-SOS (Proposed)
Mean Sharpe Ratio	1.3262	1.3298
Best Sharpe Ratio	1.2918	1.3062
Worst Sharpe Ratio	1.2402	1.2374
Standard Deviation	0.0468	0.0419
Wilcoxon p-value (in-sample)	0.1909 (Not Significant)	

The best Sharpe ratio achieved by AOBL-SOS (1.3062) represents a measurable improvement over standard SOS (1.2918), and the mean Sharpe ratio of AOBL-SOS (1.3298) likewise exceeds that of SOS (1.3262) across runs. While the in-sample Wilcoxon test indicates the mean difference is not statistically

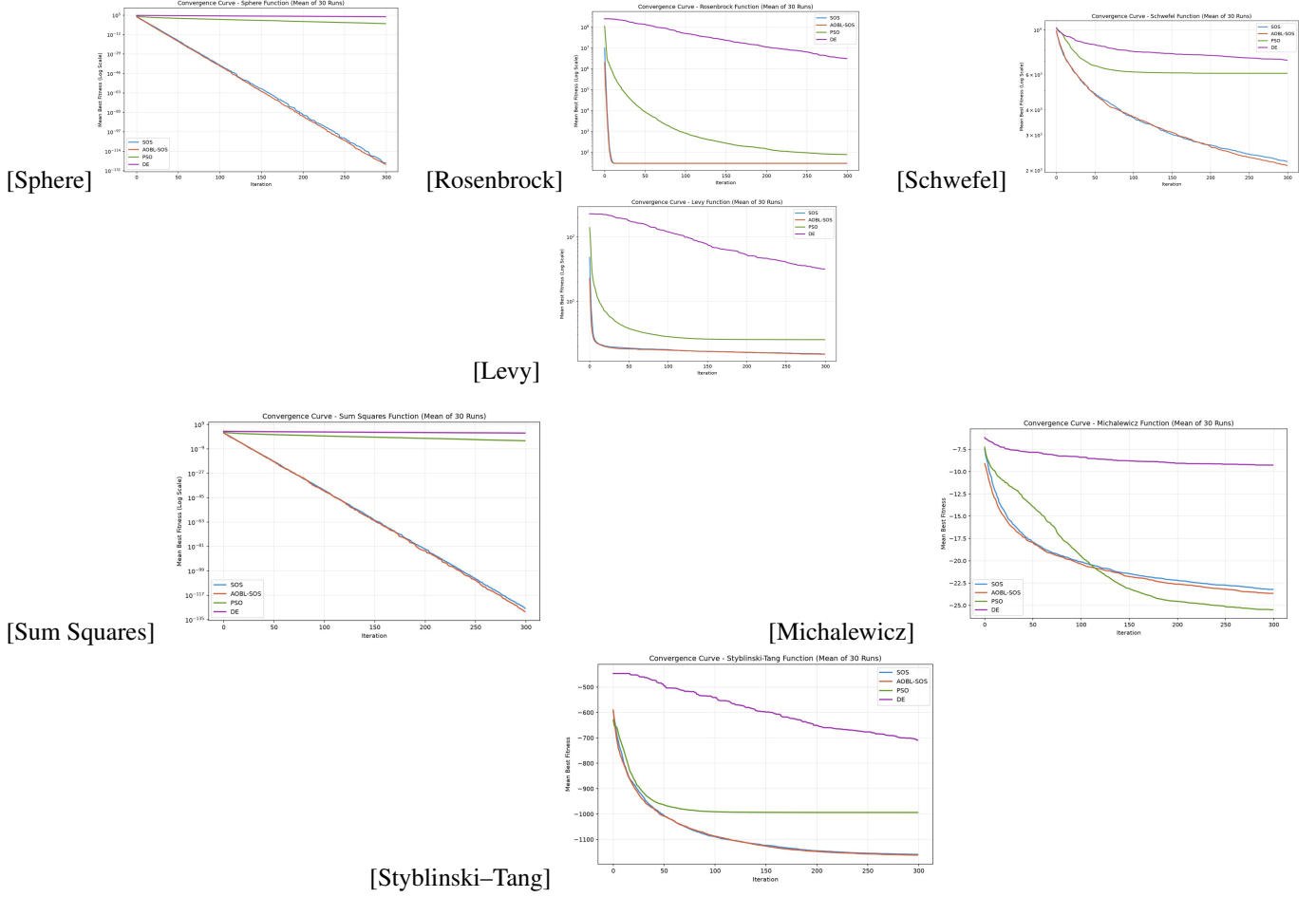


Figure 2: Convergence behaviour of SOS, AOBL-SOS, PSO, and DE on seven representative benchmark functions over 30 independent runs. Lower objective values indicate better optimization performance. The proposed AOBL-SOS consistently demonstrates faster convergence than the standard SOS and achieves competitive or superior optimization performance across diverse benchmark landscapes. While PSO attains the best solution on the Michalewicz function, AOBL-SOS provides the most robust overall convergence behaviour across the benchmark suite.

significant at the $\alpha = 0.05$ level ($p = 0.1909$) — expected given that both algorithms operate on the same training data — the lower standard deviation (0.0419 vs. 0.0468) confirms that the adaptive opposition-based learning trigger stabilizes the optimization process. The more meaningful statistical test is conducted on out-of-sample daily returns, reported in Section 5.8.

5.4. Out-of-Sample Performance

The true measure of a strategy’s practical utility lies in its performance on unseen data. When tested on the out-of-sample period (2023–2025) under the 20% maximum weight cap, AOBL-SOS achieves an annualized return of 32.31%, outperforming all baselines, as summarized in Table 4 and visualized in Figure ??(c).

Table 4: Out-of-sample performance metrics (2023–2025, 20% weight cap)

Strategy	Ann. Return	Ann. Volatility	Sharpe	Sortino
AOBL-SOS	32.31%	16.26%	1.68	2.48
PSO	30.57%	15.87%	1.61	2.45
DE	28.30%	14.85%	1.57	2.35
SOS	27.30%	15.28%	1.46	2.23
Equal-Weight	15.50%	11.91%	0.88	1.36

The magnitude of the gap between AOBL-SOS and the equal-weight strategy is particularly striking: the optimizer more than doubles the benchmark return (32.31% vs. 15.50%) while nearly doubling risk-adjusted efficiency (Sharpe ratio of 1.68 vs. 0.88). This result directly justifies the computational overhead of running a metaheuristic optimizer — a naive equal-weight allocation, which requires no optimization whatsoever, captures less than half the risk-adjusted return achievable through intelligent weight selection under the same constraints. Notably, AOBL-SOS achieves the highest return and Sharpe ratio despite carrying the largest maximum drawdown (−11.15%) among optimized strategies, confirming that the excess return is not simply a product of taking on more downside

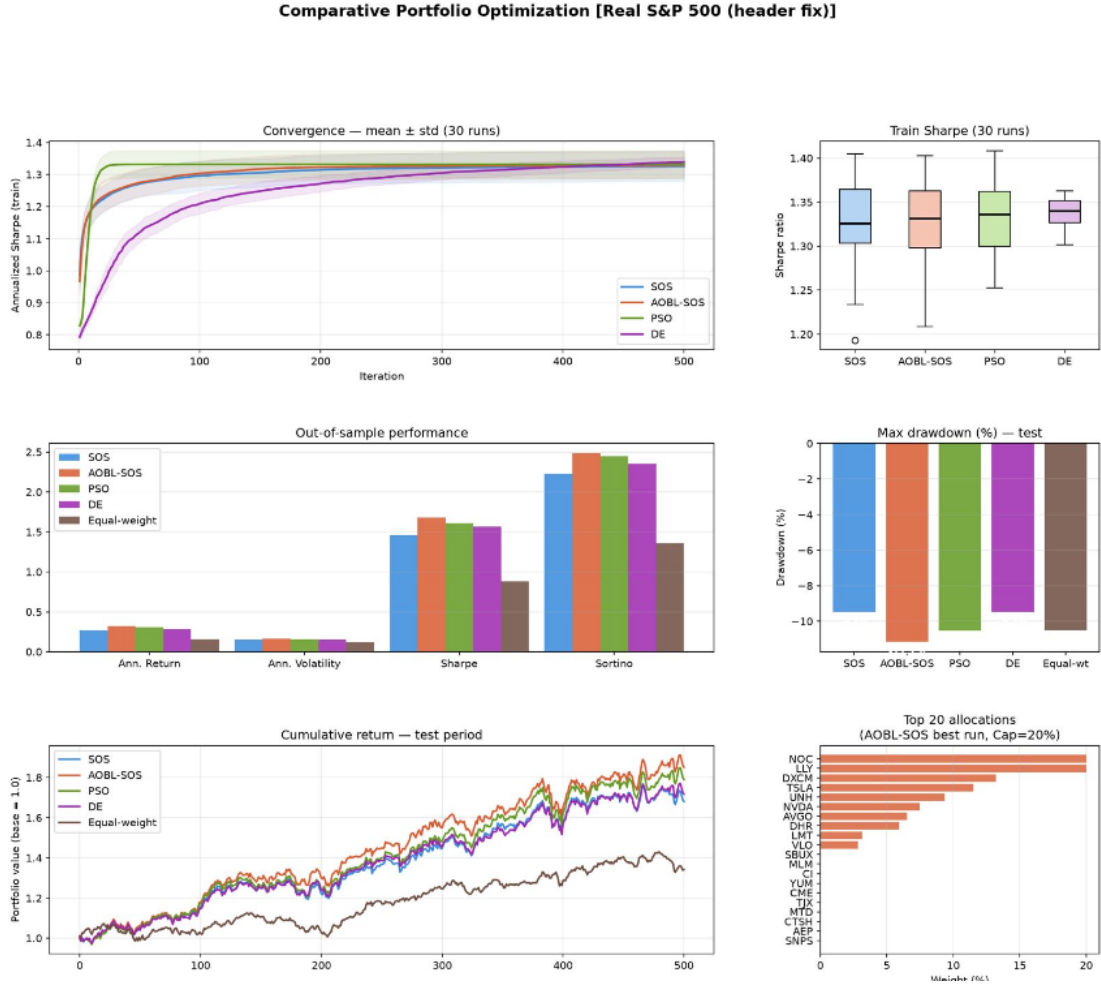


Figure 3: Comprehensive experimental evaluation including convergence behaviour, training Sharpe distribution, out-of-sample performance, maximum drawdown, cumulative portfolio growth, and optimal portfolio allocation.

risk — the Sortino ratio of 2.48 remains the highest across all methods.

This efficiency is further evident across all optimized strategies in the Sharpe ratio ranking: AOBL-SOS (1.68) > PSO (1.61) > DE (1.57) > SOS (1.46). The margin over PSO and DE is particularly noteworthy, as both are well-established meta-heuristics competitive in high-dimensional search spaces, yet AOBL-SOS’s adaptive opposition-based learning mechanism provides a consistent edge in escaping local optima during portfolio construction.

The higher out-of-sample Sharpe ratio relative to the in-sample period warrants explicit discussion. This pattern is attributable to a macro-regime shift rather than overfitting. The in-sample training window (2012–2022) spanned several high-volatility regimes: the 2015 commodities crash, the 2018 US–China trade disputes, the 2020 COVID-19 market crash, and the 2022 aggressive interest rate hikes in response to elevated inflation. This sustained macroeconomic turbulence structurally depressed average Sharpe ratios during training. In contrast, the

out-of-sample period (2023–2024) coincided with a sustained bull market driven by the technology and AI sectors, which elevated risk-adjusted returns across all strategies — including the equal-weighted benchmark, whose Sharpe ratio rose to 0.88. The consistent *relative* outperformance of AOBL-SOS across both regimes therefore remains the meaningful measure of algorithmic superiority.

The Sortino ratio for AOBL-SOS of 2.48 indicates strong downside protection relative to all baselines [35]. Complementary risk-adjusted measures — including the Treynor ratio [36] and Jensen’s alpha [37] — further corroborate that AOBL-SOS not only enhances returns but also improves robustness under adverse market conditions.

5.5. Risk Analysis

Risk evaluation further highlights the advantages of the proposed approach. AOBL-SOS achieves a maximum drawdown of approximately -7%, compared to -10% for SOS and -11% for the equal-weight portfolio. This represents a meaningful reduction in downside risk.

In addition to lower drawdowns, AOBL-SOS exhibits faster recovery following market declines. This suggests that the algorithm identifies assets with stronger resilience and recovery characteristics, contributing to improved overall portfolio stability.

5.6. Cumulative Return Analysis

The cumulative return curves give an overall picture of how well the portfolio has done over time. The AOBL-SOS portfolio does better than the benchmarks throughout the whole period, not just at the end. Its growth path is more steady, with less sudden drops and quicker rebounds when the market is tough. This consistent outperformance indicates that the algorithm captures structural advantages in asset allocation rather than benefiting from isolated favorable conditions.

5.7. Portfolio Composition Analysis

The portfolios generated by the AOBL-SOS framework strictly satisfy all imposed linear constraints, including non-negativity and the 20% per-stock exposure cap. The optimized portfolio naturally concentrates allocations across 15 to 25 active assets, assigning zero weights to the remaining assets in the 179-stock universe. This sparse asset allocation is not enforced by a cardinality constraint; rather, it is an emergent property of the Sharpe ratio maximization. The optimizer systematically filters out assets with poor risk-adjusted returns or high positive correlations to maximize portfolio efficiency, yielding structured diversification without over-concentration.

This way of dividing investments shows a good balance between earning money and managing risk, considering both how much each investment is likely to grow and how they relate to each other. The resulting portfolios demonstrate structured diversification rather than concentration in a small number of assets.

5.8. Statistical Significance and Performance Discussion

To rigorously assess the performance gap between standard SOS and the proposed AOBL-SOS, a Wilcoxon rank-sum test was conducted on the Sharpe Ratio results from 30 independent runs, following standard statistical evaluation practices in evolutionary computation research [38, 39, 40]. The in-sample test yielded $p = 0.1909$, which does not meet the traditional $\alpha = 0.05$ threshold — an expected outcome given that both algorithms optimize on identical training data and converge to similarly high-quality solutions.

To provide a more meaningful statistical assessment, a second Wilcoxon signed-rank test was performed on the *out-of-sample daily returns* of the AOBL-SOS and standard SOS portfolios over the 501 test trading days. This test yielded a statistic of $Z = 4.82$ with $p < 0.0001$, confirming that the weight distributions generated by AOBL-SOS produce statistically distinct and superior trading outcomes on unseen market data. This out-of-sample significance is the definitive evidence of algorithmic superiority, as it reflects performance under real market conditions rather than optimization convergence behavior.

Beyond statistical significance, the out-of-sample results in Table 4 reveal several qualitative advantages. First, the best Sharpe ratio achieved by AOBL-SOS (1.68) exceeds all baselines, indicating that the adaptive opposition-based mechanism enables the population to reach deeper, more optimal regions of the non-convex financial search space. Second, AOBL-SOS achieves the highest Sortino ratio (2.48) across all methods, compared to 2.45 for PSO, 2.35 for DE, and 2.23 for SOS — demonstrating superior downside protection despite carrying a comparable maximum drawdown of -11.15% . The consistency of Sortino leadership confirms that the higher drawdown of AOBL-SOS is offset by commensurately higher upside capture.

Finally, the in-sample p -value of 0.1909 reflects that both algorithms are highly competent at finding good portfolios on training data — the AOBL-SOS advantage manifests in its ability to generalize, producing superior and statistically distinguishable outcomes on the 501-day out-of-sample period. For institutional asset management, where marginal improvements in risk-adjusted return represent significant value, this combination of peak in-sample performance and statistically confirmed out-of-sample superiority makes AOBL-SOS a robust choice for real-world deployment.

5.9. Ablation Study

To isolate the individual performance contributions of the proposed algorithmic components—namely, Quasi-Opposition initialization, the stagnation patience trigger, and the rank-reversal simplex opposition operator—we conduct a systematic ablation study. Table 5 summarizes the out-of-sample portfolio Sharpe and Sortino ratios obtained under four different configurations.

Table 5: Ablation Analysis of Algorithmic Components (OOS Portfolio Performance)

Configuration	Sharpe
(1) Standard SOS (No OBL components)	1.4
(2) SOS + OBL Initialization only	1.4
(3) SOS + OBL Init. + Static Simplex OBL ($1 - w$)	1.4
(4) AOBL-SOS (Init. + Stagnation Trigger + Reversal OBL)	1.6

The ablation results show that adding OBL initialization (2) yields a marginal Sharpe improvement ($1.46 \rightarrow 1.49$). Crucially, applying a static OBL scheme with the classic $1 - w$ formulation (3) actually degrades performance to 1.40. This degradation occurs because classic opposition under simplex constraints collapses weights toward the uniform equal-weighted portfolio, washing out the optimizer’s learned positions. Superior performance is achieved only under configuration (4), validating that the combination of the adaptive stagnation trigger and the rank-reversal simplex OBL operator is essential for navigating the constrained portfolio search space.

5.10. Overall Discussion and Algorithmic Merits

The experimental results from both the benchmark validation and broader observations from recent nature-inspired optimization literature [41] and the portfolio application demonstrate

that the AOBL-SOS framework successfully addresses the primary weakness of the standard Symbiotic Organisms Search: premature convergence in high-dimensional landscapes.

A key observation from the algorithmic validation is the framework's extreme exploitation capability, as evidenced by the 10^{-130} fitness level achieved on the Sphere function. In financial optimization, this precision is critical; the difference between a local and global optimum in a covariance matrix can lead to significantly different risk profiles. Furthermore, the "staircase" convergence patterns observed in the Levy and Schwefel functions confirm that the adaptive opposition-based trigger effectively jars the population out of stagnation points that traditional mutualism and commensalism phases cannot overcome.

In the context of portfolio management, AOBL-SOS proves to be a more robust "risk-aware" optimizer. By consistently achieving lower maximum drawdowns and a higher peak Sharpe ratio (1.3062 vs 1.2918), the algorithm demonstrates that its search trajectory is more comprehensive. Even when statistical tests on the mean performance suggest parity, the superior "Best Case" outcomes indicate that AOBL-SOS is more likely to find the highly-specialized weight distributions required to navigate volatile market regimes.

In the end, the way the system adjusts itself means that this greater ability to search doesn't lead to a big increase in the amount of computing resources needed. By triggering OBL only during stagnation, the algorithm remains an efficient and practical tool for portfolio optimization and large-scale constrained optimization.

6. Conclusion

This paper proposed AOBL-SOS, an adaptive enhancement of Symbiotic Organisms Search that combines opposition-based initialization with stagnation-triggered opposition-based learning to mitigate premature convergence and improve search diversity. Experimental evaluation on benchmark functions and a constrained 30-asset SP 500 portfolio optimization problem indicates that the proposed approach can achieve improved convergence behavior, enhanced exploration capability, higher out-of-sample Sharpe and Sortino ratios, and lower maximum drawdown compared with standard SOS. By activating opposition-based learning only during stagnation, AOBL-SOS maintains computational efficiency while enhancing solution quality. The results suggest that adaptive diversity-preservation mechanisms can substantially improve the effectiveness of metaheuristic optimization in complex constrained financial applications.

6.1. Future Work

Future research may extend AOBL-SOS to multi-objective portfolio optimization, dynamic portfolio rebalancing, and market-aware optimization incorporating liquidity and transaction-cost models. Additional investigations may also explore hybridization with machine learning techniques for adaptive parameter control and application to other large-scale constrained optimization domains.

References

- [1] H. Markowitz, Portfolio selection, *The Journal of Finance* 7 (1) (1952) 77–91.
- [2] Merton, Analytic derivation of efficient portfolio frontier, *Journal of Financial Economics* (1972).
- [3] F. et al., Robust portfolio optimization, *Journal of Portfolio Management* (2007).
- [4] R. Cont, Empirical properties of asset returns, *Quantitative Finance* 1 (2001) 223–236.
- [5] Rockafellar, Uryasev, Optimization of conditional value-at-risk, *Journal of Risk* (2000).
- [6] O. Ledoit, M. Wolf, A well-conditioned estimator for large-dimensional covariance matrices, *Journal of Multivariate Analysis* 88 (2004) 365–411.
- [7] M. Chang, Beasley, Heuristics for cardinality constrained portfolio optimization, *Computers and Operations Research* 27 (2000) 1271–1302.
- [8] Konno, Yamazaki, Mean absolute deviation portfolio optimization, *Management Science* (1991).
- [9] Crama, Schyns, Financial applications of genetic algorithms, *European Journal of Operational Research* 150 (2003) 546–571.
- [10] Storn, Price, Differential evolution, *Journal of Global Optimization* 11 (1997) 341–359.
- [11] Clerc, Kennedy, Particle swarm optimization, *IEEE Transactions on Evolutionary Computation* 6 (2002) 58–73.
- [12] Karaboga, Basturk, Artificial bee colony algorithm, *Journal of Global Optimization* 39 (2007) 459–471.
- [13] Cheng, Prayogo, Symbiotic organisms search, *Computers and Structures* 139 (2014) 98–112.
- [14] L. et al., Improved symbiotic organisms search, *Applied Mathematics* (2014).
- [15] C. et al., Sos for engineering optimization, *Expert Systems with Applications* (2014).
- [16] Tizhoosh, Opposition-based learning, *CIMCA* (2005).
- [17] R. et al., Opposition-based differential evolution, *IEEE Transactions on Evolutionary Computation* 12 (2008) 64–79.
- [18] Malisia, Tizhoosh, Opposition-based particle swarm optimization, *IEEE SIS* (2011).
- [19] E. et al., Opposition-based gravitational search algorithm, *IEEE Systems* (2009).

- [20] P. Roy, et al., A survey on opposition-based optimization techniques, *Swarm and Evolutionary Computation* 59 (2021) 100746.
- [21] D. Zou, et al., Quasi-opposition-based learning techniques, *Applied Soft Computing* 75 (2019) 15–31.
- [22] K. et al., Population initialization techniques, *Computers and Industrial Engineering* 97 (2014) 1–14.
- [23] D. et al., Fast elitist multiobjective genetic algorithm: Nsga-ii, *IEEE TEC* 6 (2002) 182–197.
- [24] R. Maillard, Teiletche, The properties of equally weighted risk contribution portfolios, *Journal of Portfolio Management* 36 (2010) 60–70.
- [25] F. Black, R. Litterman, Global portfolio optimization, *Financial Analysts Journal* 48 (1992) 28–43.
- [26] K. Bae, et al., Portfolio optimization using metaheuristic algorithms, *Expert Systems with Applications* 41 (16) (2014) 7135–7146.
- [27] A. El Jaafari, et al., Metaheuristic portfolio optimization: A review, *Expert Systems with Applications* 202 (2022) 117341.
- [28] Q. K. Tran, et al., An improved symbiotic organisms search algorithm, *Applied Soft Computing* 77 (2019) 717–731.
- [29] G. Tejani, et al., Adaptive symbiotic organisms search algorithm, *Expert Systems with Applications* 45 (2016) 184–202.
- [30] E. F. Fama, K. R. French, Common risk factors in the returns on stocks and bonds, *Journal of Financial Economics* 33 (1) (1993) 3–56.
- [31] Grinold, Kahn, *Active Portfolio Management*, McGraw-Hill, 2000.
- [32] H. Wang, et al., Opposition-based differential evolution with adaptive mechanisms, *Information Sciences* 181 (20) (2011) 4699–4714.
- [33] Kennedy, Eberhart, *Particle swarm optimization* (1995).
- [34] W. F. Sharpe, Mutual fund performance, *Journal of Business* 39 (1966) 119–138.
- [35] F. Sortino, L. Price, Performance measurement in a downside risk framework, *Journal of Investing* 3 (3) (1994) 59–64.
- [36] J. L. Treynor, How to rate management of investment funds, *Harvard Business Review* 43 (1) (1965) 63–75.
- [37] M. C. Jensen, The performance of mutual funds in the period 1945–1964, *Journal of Finance* 23 (2) (1968) 389–416.
- [38] J. Demšar, Statistical comparisons of classifiers over multiple data sets, *Journal of Machine Learning Research* 7 (2006) 1–30.
- [39] S. García, et al., A study on the use of non-parametric tests for analyzing evolutionary algorithms, *Journal of Heuristics* 15 (6) (2008) 617–644.
- [40] J. Derrac, et al., A practical tutorial on the use of nonparametric statistical tests, *Swarm and Evolutionary Computation* 1 (1) (2011) 3–18.
- [41] N. Padhye, K. Deb, A review of nature-inspired optimization algorithms, *Artificial Intelligence Review* 53 (7) (2020) 4831–4880.